


MVC Architecture ↴

(11:50)

M → Models

V → views

C → Controllers

We follow MVC architecture → taaki particular folder mei usi chiz se related code/ other files ho. (Separation of Concern) and also to follow DRY principle taaki code repeat na ho.

schemas → models folder mei banenge

(.ejs) templates → views

routes/ api → Routes

" " "

We are creating monolithic application using MVC architecture.

Step 1 → npm init -y

Step 2 → npm i express nodemon ejs mongoose

Step 3 → touch app.js

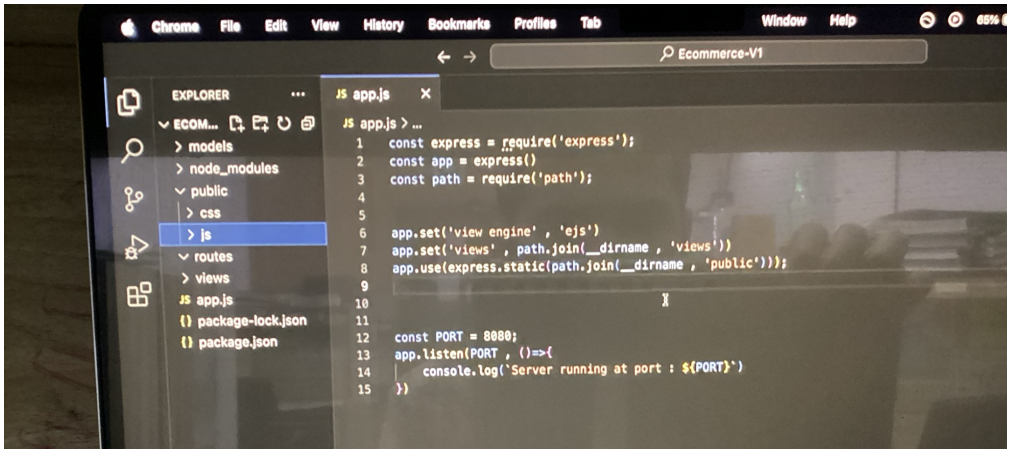
Step 4 → mkdir public views routes models

↓
static
files ke
liye

↓
.ejs

yaha tak hamne folders set kar diye.

Step 1 iske baad hum server banayenge, we will write this code :-



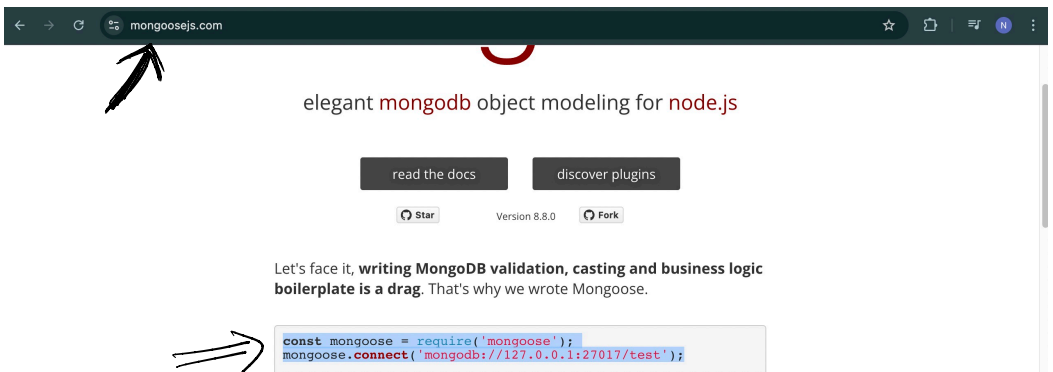
```
1 const express = require('express');
2 const app = express();
3 const path = require('path');
4
5
6 app.set('view engine', 'ejs');
7 app.set('views', path.join(__dirname, 'views'));
8 app.use(express.static(path.join(__dirname, 'public')));
9
10
11
12 const PORT = 8080;
13 app.listen(PORT, () => {
14   console.log('Server running at port : ${PORT}')
15 })
```

Step 2:- Mongoose Connection Karo :-

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/test');
```

mongoosejs.com

add it in your code.



```
mongoose.connect('mongodb://127.0.0.1:27017/test')
```

→ this line returns promise, that's why we will use `.then()` `.catch()` to handle the promise.

```
const express = require('express');  
const app = express();  
const path = require('path');  
const mongoose = require('mongoose');
```

```
app.set('view engine', 'ejs');  
app.set('views', path.join(__dirname, 'views'));  
app.use(express.static(path.join(__dirname, 'public')));
```

```
mongoose.connect('mongodb://127.0.0.1:27017/nov_batch')  
  .then(()=>{console.log('DB connected')})  
  .catch((err)=>{console.log("Error is: ", err)})
```

```
const PORT = 8080;  
app.listen(PORT, ()=>{  
  console.log(`Server running at port : ${PORT}`)  
});
```

→ this is our database name.

ab nodemon app.js ya

npm start likho

to check if server is working or not.

Step 3: Ab aapko schema banana padega. Schema code is written inside Model folder. So, create a .js file in it and write code inside it.

Schema = Blueprint of document

Schema hum Models folder mei banate hai.

Samarth sir ka convention hai ki model folder ke .js file ka first letter hamesha capital hoga.

Product.js

↳ created inside models folder.

we will create ^a model inside Product.js file using mongoose.

Isthiye pehle mongoose require karo.

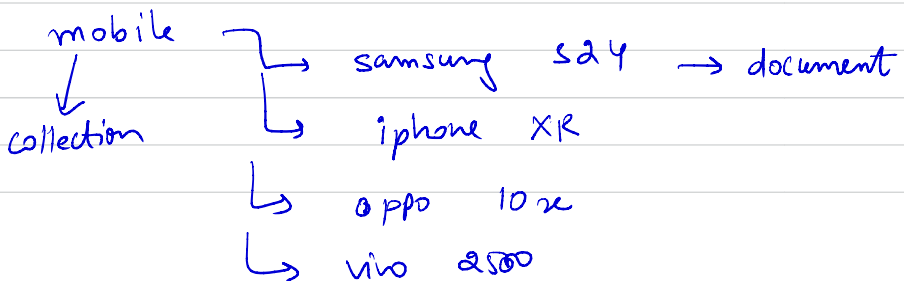
Ab hum schema banayenge. go to mongoose documentation to learn about creating schema.

jab aap ecommerce website khologe, you will see one column of mobile, one of laptop, one of mouse.

mobile, laptop, mouse

↳ they are collections.

informally, you can call them category.



name to hoga hi.

this is a schema.
isme apke document
ki property / information
hoti hai.

→ २८ नो

trim to remove space $\Rightarrow$ Nitin

if you want

provide you info and dont want him to skip it.

age and pichhe ke
space remove karne
ke liye

```
const mongoose = require('mongoose');
```

```
const productSchema = new mongoose.Schema({  
  name: {  
    type: String,  
    trim: true,  
    required: true  
  },  
  
  img: {  
    type: String,  
    trim: true,  
  },  
  
  price: {  
    type: Number,  
    min: 0,  
    required: true  
  },  
  
  desc: {  
    type: String,  
    trim: true  
  },  
});
```

→ yaha schema
banaya

```
let Product = mongoose.model('Product', productSchema);  
  
module.exports = Product;
```

→ yaha model banaya

Product.js of models folder
↓
name of ~~the~~ collection/ model ⇒ Product

Note:- model is a JS class and Class se object
banate hai and vo object hi document hai.

modelName should be singular with first letter
capital always.

let Product = mongoose.model('Product', productSchema);
↓
Jaisi nahi ki dono same
name use kar sakte hai. Agle
naam use kar sakte hai

let Product

↳ This will be used in VSCode to use it

```
mongoose.model("Product" ...)
```



ye name se database mei model/collection banega.

this is a JS class.

Now we will export Product using
`module.export = Product;`

(45:00 to 58)

7 Nov, 2024

Ab hum documenter banayenge ⇒ we will send data in array using `insertMany()`. Array mei mai kaafi sare saman ek saath bhejunga. Kafi sare saman ko ek saath bhejne ka process ko seed khte hai. Now we will make a `seed.js` file.

Ab mai `seed.js` file banaunga inside my ecommerce V1 folder. (separation of concern)

seed.js mei mera array hoga jise mai insert karunga in database.

database seeding:- jab mujhe chije ko ek saath database mei bhejna hai. That is database seeding.

Ab mujhe jis collection / Model mei array chahiye, mujhe vo bhi to chahiye hoga.

Let's create a function first to push elements / documents in model / collection

Jaise hi mai "Product" type karunga code mei, ye line khud se add ho jayegi.

```
const Product = require('./models/Products');
```

insertMany() will return a promise and use handle ke liye, we ~~are~~ will use async await.

```
async function seedDB(){  
  await Product.insertMany(products);  
  console.log("DB seeded")  
}
```

Note:- Sabse pehle hamne seed.js mei mongoose require kiya hai.

```

const mongoose = require('mongoose');
const Product = require('./models/Products');

const products = [
  {
    name: "iphone 15pro",
    img: "https://images.unsplash.com/photo-1695048133142-1a20484d2569?w=800&auto=format&fit=crop&q=60&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxzZWZlY2h8M3x8aXBob25lJTIwMTV8ZW58MHx8MHx8fDA%3D",
    price: 124000,
    desc: "very costly phone it is"
  },
  {
    name: "macbook pro",
    img: "https://images.unsplash.com/photo-1598495037740-2c360cf49e50?w=800&auto=format&fit=crop&q=60&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxzZWZlY2h8M3x8aXBob25lJTIwMTV8ZW58MHx8MHx8fDA%3D",
    price: 230000,
    desc: "hello i am a good machine"
  },
  {
    name: "apple pencil",
    img: "https://images.unsplash.com/photo-1551651639-927b595f815c?w=800&auto=format&fit=crop&q=60&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxzZWZlY2h8M3x8aXBob25lJTIwMTV8ZW58MHx8MHx8fDA%3D",
    price: 10000,
    desc: "i can write future"
  }
]

async function seedDB(){
  await Product.insertMany(products);
  console.log("DB seeded")
}

module.exports = seedDB;

```

→ i will export seedDB function taaki mai use use kar pau jaha bhi mujhe jarurat hogi. Mujhe app.js mei seedDB chahiye hoga.

app.js mei seedDB likhne se automatically ye line add ho jayegi top par,

```
const seedDB = require('./seed');
```

seed DB() ko comment kar dena when not needed. otherwise repeatedly same data add ho jayega.

Ab mai nodemon app.js karunga, then I will be able to see inserted data in database using terminal.

```
(test> show dbs
admin          40.00 KiB
amazonDb       72.00 KiB
asdfg          8.00 KiB
config         72.00 KiB
local          80.00 KiB
movieApp       72.00 KiB
nov-batch      40.00 KiB
sachin         8.00 KiB
samarth        72.00 KiB
(test> use nov-batch
switched to db nov-batch
nov-batch> show collections
products
nov-batch> db.products.find()
[
  {
    _id: ObjectId('672c63f01d2bc4480758af3c'),
    name: 'iphone 15pro',
    img: 'https://images.unsplash.com/photo-1695048133142-1a20484d2569?w=800&auto=format&fit=crop&q=60&iqlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxWMFk5Y2h8M3x8YXB0b25lJTlWMTV8ZW58MHx8MHx8fDA%3D',
    price: 124000,
    desc: 'very costly phone it is',
    __v: 0
  },
  {
    _id: ObjectId('672c63f01d2bc4480758af3d'),
    name: 'macbook pro',
    img: 'https://images.unsplash.com/photo-1598495037740-2c360cf49e50?w=800&auto=format&fit=crop&q=60&iqlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxWMFk5Y2h8Nnx8bWV8ZjJlYW9vayUyMHByb3x1bnw8fHw8fHx8MA%3D%3D',
    price: 230000,
    desc: 'hello i am a good machine',
    __v: 0
  },
  {
    _id: ObjectId('672c63f01d2bc4480758af3e'),
    name: 'apple pencil',
    img: 'https://images.unsplash.com/photo-1551651639-927b595f815c?w=800&auto=format&fit=crop&q=60&iqlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxWMFk5Y2h8M3x8YXB0bGU1MjBwZW5jaWw8ZW58MHx8MHx8fDA%3D',
    price: 10000,
    desc: 'i can write future',
    __v: 0
  }
]
nov-batch>
```

inserted data

Break over at (1:32:00)

Ab tak hum model/ collection mei data seed kar chuke hai.

Now I would like to see data (inside array) on web page. Ab hum routes banayenge.

We will be making different routes. For example one for Add To Cart, one for product, one for ~~user~~ user, etc.

Routes folder mei hum $\rightarrow$ product.js bananaenge.
model folder mei we made $\rightarrow$ Product.js
routes folder mei we made $\rightarrow$ product.js
spellings are different.

Ab kyunki hum routes bananaenge inside routes folder — product.js, we will need express server.

So, we will require express, then we will create mini application in product.js.
We cannot export/import const app of app.js as app is an instance.

Iss mini application ko hum router kehate hai.

```
const router = express.Router();
```

$\hookrightarrow$ this is mini application.

jo Kaam hum app.get() se kar rahe the,
ab wo router.get() se karenge.

Ab hum CRUD perform karenge:-

Route name	URL	HTTP Verb	Description
Index .ejs	/blogs	GET	Display a list of all blogs task 1
New .ejs	/blog/new	GET	Show form to make new blogs 2
Create	/blogs	POST	Add new blog to database, then <u>redirect</u> 3
Show .ejs	/blogs/:id	GET	Show info about one blogs 4
Edit .ejs	/blogs/:id/edit	GET	Show edit form of one blog 5
Update	/blogs/:id	PUT	Update a particular blog, then <u>redirect</u> 6
Destroy	/blogs/:id	DELETE	Delete a particular blog, then <u>redirect</u> 7

we will create index.ejs, new.ejs, show.ejs, edit.ejs in views folder.

Jaha jaha redirect hai hum uska .ejs file nahi banayenge. For example: update.ejs, destroy.ejs

The process is same like we did earlier.

Task 1 → display list of all products/documents. Hamare documents saved kaha hai? They are inside a collection/model named Product (in Product.js file of models folder). Isliye main Product ko require Kiya (1), then 2 par hamne Product.find(2) use karke return ki hui value products variable mei store kara di.

Remaining things are same like we studied earlier in CRUD. 2 promise return karega isliye await async use Kiya hamne.

This is the code of task 1 and task 2.

```
const express = require('express');
const Product = require('../models/Products'); → l1

const router = express.Router();

// display all documents

router.get('/products', async (req, res) => {
  let products = await Product.find({});
  res.render('index', {products}); → l2
})

// create new document

router.get('/products/new', (req, res) => { → l4
  res.render('new');
})

module.exports = router; → l3
```

↳ products.js
(routes folder)

l3 → explained on next page.

l4 is for task 2 → to edit the document.

```
<h1>all documents</h1>

<% for(let item of products){ %>
  <h1> <%=item.name%> </h1>
  
  <h2> <%=item.price%></h2>
  <p> <%=item.desc%></p>

<% } %>
```

⇒ index.ejs

```
<h1>new product form</h1>
```

```
<form action="">
  <label for="naam">Name:</label>
  <input type="text" id="naam" name="name">

  <br>
  <br>

  <label for="pic">Image:</label>
  <input type="text" id="pic" image="image">

  <br>
  <br>

  <label for="desc">Name:</label>
  <input type="text" id="desc" desc="desc">

  <button type="submit">Add Product</button>
</form>
```

⇒ new.ejs

Explanation of l3 :-

2. Why export router ?

- When you export `router` from `product.js`, it allows other files to access and use it.
- In your `app.js`, you import this `router` with `const productRoutes = require('./routes/product');`. Then, you add it to your app with `app.use(productRoutes);`.
- By doing this, all routes defined in `product.js` (`/products` and `/products/new`) are registered to your main Express app, making them accessible when users visit those routes.

3. What if you don't export router ?

- If you don't export `router`, `app.js` won't have access to any routes defined in `product.js`.
- Without `app.use(productRoutes);` in `app.js`, the routes in `product.js` won't be added to your main app, so visiting `/products` or `/products/new` would result in 404 errors.

app.js :-

```
const express = require('express');
const app = express();
const path = require('path');
const mongoose = require('mongoose');
const seedDB = require('./seed');
const productRoutes = require('./routes/product');

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.static(path.join(__dirname, 'public')));

// seedDB()
app.use(productRoutes);
mongoose.connect('mongodb://127.0.0.1:27017/nov_batch')
  .then(() => { console.log('DB connected') })
  .catch((err) => { console.log("Error is: ", err) });

const PORT = 8080;
app.listen(PORT, () => {
  console.log(`Server running at port : ${PORT}`)
});
```

Task 3 - Add new document to collection, then redirect.

```
router.post('/products', async (req, res) => {
  let { name, img, price, desc } = req.body; // Now all fields will
  be defined
  await Product.create({ name, img, price, desc }); // Automatically
  saved to DB
  res.redirect('/products');
});
```

→ product.js

Product.create → to create new document inside collection named Product.

Product.create will return a promise like use handle kame ke liye async await use hua hai.

```
<form action="/products" method="post">
  <label for="naam">Name:</label>
  <input type="text" id="naam" name="name">

  <br><br>

  <label for="pic">Image:</label>
  <input type="text" id="pic" name="img">

  <br><br>

  <label for="price">Price:</label>
  <input type="text" id="price" name="price">

  <br><br>

  <label for="desc">Description:</label>
  <input type="text" id="desc" name="desc">

  <br><br>

  <button type="submit">Add Product</button>
</form>
```

↳ new.ejs

```
app.use(express.urlencoded({extended:true}))
```

⇒ added in app.js

we also added middle ware function in app.js
because by default req.body is undefined.

Task 4 :- Show info about one document :-

(02:04:20)

```
router.get('/products/:id', async (req, res) =>{  
  let {id} = req.params;  
  let foundProduct= await Product.findById(id);  
  res.render('show' , {foundProduct} );  
})
```

⇒ l 2

→ product.js

```
<h1>Displaying a particular product</h1>
```

```
<div>
```

```
  <h1><%=foundProduct.name%></h1>
```

```
  
```

```
  <h3><%=foundProduct.price%></h3>
```

```
  <p><%=foundProduct.desc%></p>
```

```
</div>
```

→ show .js

```
<h1>all documents</h1>
```

```
<% for(let item of products){ %>
```

```
  <h1> <%=item.name%> </h1>
```

```
  
```

```
  <h2> <%=item.price%></h2>
```

```
  <p> <%=item.desc%></p>
```

```
l1 <a href="/products/<%=item._id%>">Show this</a>
```

```
<% } %>
```

→ index.
ejs

Hamne index.ejs mei l1 add kari, ab mei jab show this par click karunga, it is a get request.

Now l2 of product.js will work

Product.findById () ⇒ we got this mongoosejs.com documentation.

Task 5 :- Show edit form :-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

  <h1>Edit Product Form</h1>

  <form action="" method="">
    <div>
      <label for="naam">Name: </label>
      <input type="text" name="name" id="naam" value="<%=foundProduct.name%>">
    </div>
    <div>
      <label for="pic">Image: </label>
      <input type="text" name="img" id="pic" value="<%=foundProduct.img%>">
    </div>
    <div>
      <label for="daam">Price: </label>
      <input type="text" name="price" id="daam" value="<%=foundProduct.price%>">
    </div>
    <div>
      <label for="des">Description: </label>
      <textarea name="desc" id="desc" rows="3"> <%=foundProduct.desc%> </textarea>
    </div>
    <button type="submit">Edit Product</button>
  </form>
</body>
</html>
```

→ edit.ejs

```
<h1>all documents</h1>

<% for(let item of products){ %>
  <h1> <%=item.name%> </h1>
  
  <h2> <%=item.price%></h2>
  <p> <%=item.desc%></p>

  <a href="/products/<%=item._id%>">Show this</a>
  <br>
  <a href="/products/<%=item._id%>/edit">Edit This</a>

<% } %>
```

→ index.ejs

```
router.get('/products/:id/edit' , async(req,res)=>{
  let {id} = req.params;
  let foundProduct = await Product.findById(id);
  res.render('edit' , {foundProduct})
})
```

→ product.js

Task 4 jaisa hisame hai . Hamne index.ejs mei extra line add kar di.

Task 6:- update particular ~~can~~ document, then redirect.

```
const methodOverride = require('method-override');  
app.use(methodOverride('_method'));
```

⇒ we added them in app.js

```
<h1>Edit Product Form</h1>
```

```
  <form action="/products/<%=foundProduct._id%>?_method=PATCH"  
method="POST">  
    <div>  
      <label for="naam">Name: </label>  
      <input type="text" name="name" id="naam"  
value="<%=foundProduct.name%>">  
    </div>  
    <div>  
      <label for="pic">Image: </label>  
      <input type="text" name="img" id="pic"  
value="<%=foundProduct.img%>">  
    </div>  
    <div>  
      <label for="daam">Price: </label>  
      <input type="text" name="price" id="daam"  
value="<%=foundProduct.price%>">  
    </div>  
    <div>  
      <label for="des">Description: </label>  
      <textarea name="desc" id="desc" rows="3"> <%=foundProduct.desc%>  
</textarea>  
    </div>  
    <button type="submit">Edit Product</button>  
</form>
```

↳ edit.ejs

<input> ⇒ iska ek attribute hai value ⇒ we will use it to show the key values of foundProduct object.

But this thing is not needed for <textarea>.


```
router.patch('/products/:id' , async(req,res)=>{  
  let {id} = req.params;  
  let {name , img , price , desc} = req.body;  
  await Product.findByIdAndUpdate(id , {name , img , price , desc} );  
  res.redirect('/products')  
})
```

→ product.js
→ we used it to find document by Id.

Task 7 :- Delete a document :-

```
router.delete('/products/:id' , async(req,res)=>{  
  let {id} = req.params;  
  await Product.findByIdAndDelete(id);  
  res.redirect('/products')  
})
```

→ product.js

```
<form action="/products/<%=item_id%>?_method=DELETE" method="POST">  
  <button>Delete</button>  
</form>
```

→ index.ejs

